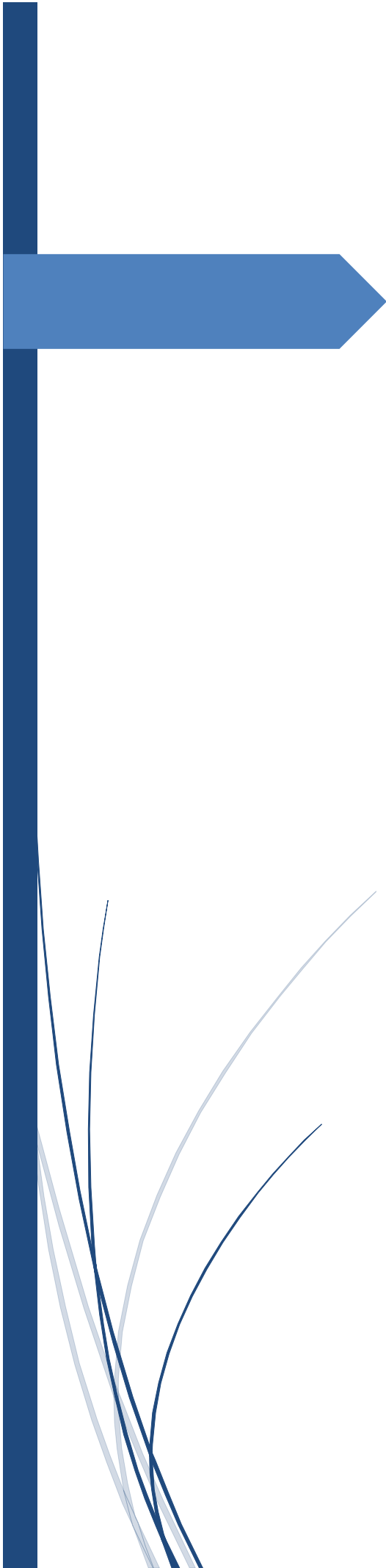




Desayuno - CafeHuevos

Asincronía, Paralelismo y Concurrency

Álvaro Ruiz Velázquez
DWES - 2ºDAW

1. ¿Qué diferencias has observado entre las 5 soluciones?

- Síncrono: una única tarea detrás de otra; el programa permanece bloqueado.
 - AsíncronoMal: utiliza Task, pero sigue siendo secuencial porque cada await espera antes de iniciar la siguiente operación.
 - AsíncronoPro: organiza las tareas independientes para que trabajen en paralelo con Task.WhenAll.
 - AsíncronoMal + CancellationToken: igual que AsíncronoMal pero con cancelación por tiempo.
 - AsíncronoPro + CancellationToken: igual que AsíncronoPro pero con cancelación por tiempo.
-

2. ¿Qué acciones se pueden ejecutar a la vez y cuáles no? ¿Por qué?

Se pueden ejecutar a la vez	Porque...
Preparar café	No depende de ninguna otra tarea.
Calentar sartén	No depende de ninguna otra tarea.
Tostar pan	No depende de ninguna otra tarea.
Preparar zumo	No depende de ninguna otra tarea.

Se pueden ejecutar a la vez, pero después de las anteriores	Porque...
Untar mermelada	Necesita que el pan ya esté tostado.
Freír huevo	Requiere que la sartén esté caliente.
Freír beicon	Requiere que la sartén esté caliente.

- El huevo y el beicon se pueden hacer a la vez, puesto que asumimos que contamos con una sartén lo suficientemente grande como para poder hacer las dos cosas a la vez
-

3. ¿Qué ha pasado con cada solución cuando introduces el timeout?

- Síncrono: no admite cancelación porque utiliza llamadas bloqueantes (Thread.Sleep).
- AsíncronoMal: la cancelación ocurre a los 500ms, durante FreirHuevo, por lo que deja el resto sin completar, puesto que funciona como el Síncrono, con llamadas bloqueantes.

- AsíncronoPro: todas las tareas empiezan antes, pero las que aún no han terminado (FreirHuevo + FreirBacon + UnterMermelada) reciben el CancellationToken y lanzan OperationCanceledException cuando el tiempo expira.
-

4. ¿El enfoque con mejor rendimiento es también el más seguro? ¿Por qué?

- No siempre.
 - La solución asíncrona pro es más rápida porque permite realizar varias tareas al mismo tiempo. Sin embargo, también es un poco más complicada y hay que tener cuidado con el orden de las tareas para evitar errores. En este ejercicio funciona bien porque las tareas no dependen unas de otras, excepto en los casos en los que sí hemos establecido un orden, como tostar el pan antes de untar la mermelada. En otros programas, hacer muchas tareas a la vez podría provocar problemas.
-

5. ¿Merece la pena complicarse con paralelismo o con mecanismos de control de tiempo? Justifica tu respuesta.

- El uso de paralelismo es beneficioso cuando hay operaciones que pueden ejecutarse de manera independiente, ya que, en tales situaciones, el tiempo de ejecución se reduce de 1500 ms a 500 ms.
- Los mecanismos de timeout también resultan ser prácticos, ya que impiden que una aplicación se quede esperando de forma indefinida una operación que sea lenta. Aunque esto añade complejidad, contribuye a que el programa sea más sólido y que responda de manera más efectiva ante bloqueos o retrasos.